Static ExpoComm on Adversarial Pursuit

We re-train the released *static* ExpoComm agent of Li et al. [1] on three Adversarial Pursuit scenarios with training seed 1 and the paper's 5M-step budget. Final 100-episode returns of 69.1, 61.3 and 69.2 (25, 45, 61 agents) exceed the paper's traced five-seed static means. **Trained predators herd prey against the boundary and tag them repeatedly.** In fixed probe episodes, about 97% of tag attempts earn a reward. These single-seed results demonstrate task learning; the benefit of communication remains untested.

1 Experimental setup

We use the released agent [2] unchanged: independent Q-learning agents exchanging messages over a static exponential graph with $K = \lceil log_2 N \rceil$ peers (5, 6, 6 for $N = 25, 45, 61$) and a global-state prediction auxiliary loss (weight 0.1). Hyper-parameters follow Table 2 and App. B.1 of the paper: hidden size 64, $\gamma = 0.99$, batch 32, learning rate 5×10^{-4}, replay buffer of 2,000 episodes and 5M environment steps; ε anneals to the released configuration's 0.05 over 0.5M steps. Maps are 45×45, 60×60 and 70×70 with 25/45/61 learning predators against twice as many pretrained prey; a successful tag yields $+1$ and each tag attempt costs -0.2. Every 50k steps the greedy policy plays 100 evaluation episodes of 200 steps; we report the mean over episodes of the undiscounted return averaged over predators, always at the last of the 101 evaluations (step 5,000,200; no checkpoint selection). Runs used one A100 (80 GB; a 40 GB MIG slice for $N = 45$) for 22.8, 36.8 and 46.3 h.

2 Results

Final returns exceed the traced paper means by 18, 35 and 40 units; their terminal levels are first reached within 18–28% of the training budget. Over the last 2M steps, return slopes are +3.8, +3.4 and +11.6 per 1M steps for 25, 45 and 61 agents: the largest team is still improving.

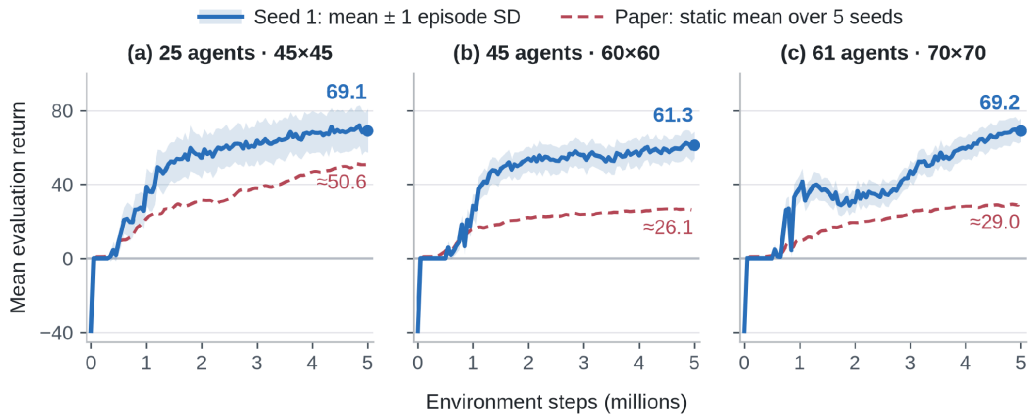


Figure 1: Seed-1 learning across all three team sizes. Blue: unsmoothed means over 100 greedy episodes with $\pm$ 1 episode SD; red dashed: the paper's traced five-seed static mean from Fig. 4(a–c) (ending near 4.95M). Shading describes variability across evaluation episodes, not training seeds.

Table 1: Final returns (mean $\pm$ episode SD; step 5,000,200) and learning speed. Paper values: traced five-seed static means. Agent counts link to W&B.

Agents	Evaluation return			Steps to paper level (M)	
	Final ± SD	Last 5	Paper ≈	First	Sustained
25	**69.1 ± 11.9**	69.6	50.6	1.40	1.40
45	**61.3 ± 7.2**	61.2	26.1	1.00	1.00
61	**69.2 ± 6.1**	69.0	29.0	0.90	1.95

Last 5: mean of the last five evaluations. *Sustained*: every later evaluation remains at or above the paper's terminal mean. Times are in millions of environment steps.

3 Boundary herding and repeated tagging

Trained predators herd prey against the boundary and tag them repeatedly. Figure 2 follows one trained 25-agent episode: predators start dispersed, gather along the lower edge, and keep tagging nearby prey. The connected close-ups show the same patch at both later times.

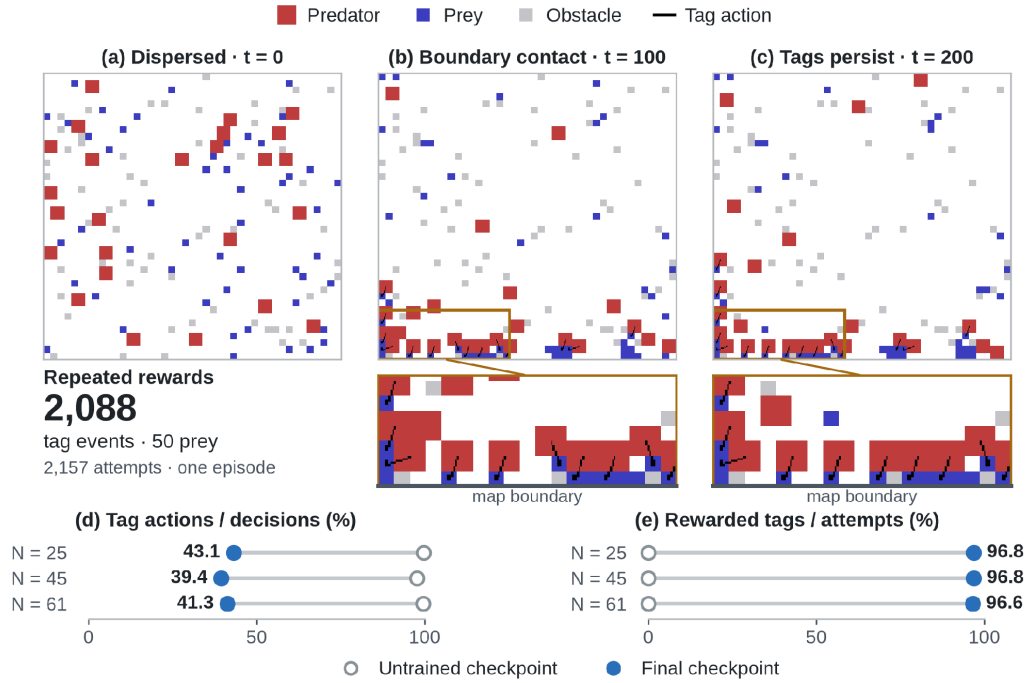


Figure 2: Boundary trapping persists. (a–c) One fixed 25-agent episode of the final seed-1 policy at $t = 0$, 100, 200; orange boxes connect to 2.3× enlargements. Episode return: 66.3 (separate 100-episode mean: 69.1). (d, e) One matched probe per team size, comparing untrained (200 steps) and final (5,000,200 steps) checkpoints. Rewarded tag events are inferred from return and attempts; they are not unique prey captures.

The return comes from accurate, repeated tags. Across the three probes, training reduces tag actions from 98–100% to 39–43% of decisions, while 96.6–96.8% of trained attempts earn a reward. For $N = 25$, 2,088 rewarded tag events from 2,157 attempts yield 66.3 return per predator after the tag penalty. There are only 50 prey: the reward counts repeated tag events. The matched untrained policy earns just one rewarded tag and returns -39.9. The policy learns *when and where* to tag.

4 Discussion and limitations

Single-seed scope. We report training seed 1 only. Compute and runtime constraints limited seed coverage, with each run taking 22.8–46.3 hours. Its returns exceed the paper's traced static means, but one training seed cannot establish superiority over a five-seed average or estimate training variability. The probes cover one fixed map per team size; they do not measure how often boundary herding occurs across evaluation episodes or establish deliberate coordination.

Task reward is not evidence for communication. In the released static agent the attention keys and values are built only from the previous messages, which start at zero and are shared, so every agent aggregates the same observation-independent message by construction; a six-step synthetic probe that perturbed one agent's observation on the frozen 25-agent checkpoint moved its own Q-values by up to 8.4 but its peers' by at most 1.5×10^{-5} in float32 (2.8×10^{-14} in float64), i.e. float noise. The released static agent is thus effectively independent DQN: our returns show that it learns the task, not that exponential-topology communication causes it; a no-communication control with the same seed is the natural next experiment. *Scope:* static variant only (no one-peer variant, baselines, Battle or IMP); all 303 scheduled evaluation means match the archived logs and CSV exports. Data and figure code accompany this report.

References

[1] X. Li, X. Wang, C. Bai, and J. Zhang. Exponential topology-enabled scalable communication in multi-agent reinforcement learning. In *ICLR*, 2025. arXiv:2502.19717.

[2] X. Li, X. Wang, C. Bai, and J. Zhang. ExpoComm code, rev. `ba63ecd`. github.com/LXXXXR/ExpoComm.

Wandb links: 25 agents 45 agents 61 agents

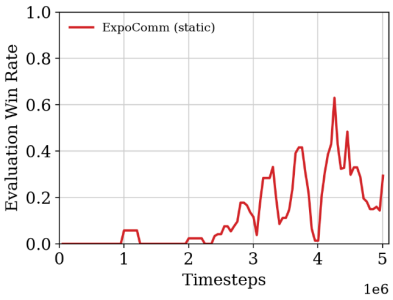
Results

All runs finished 5.05 million steps. The learned (red) team's evaluation win rate ended at 0.29 with 20 agents, 0.53 with 42 agents and 0.996 with 64 agents (Table 2). Because a single end-of-training value is noisy, we also report the mean of the last ten evaluations. Mean return is on a much smaller scale than win rate and stayed close to zero for 20 and 42 agents.

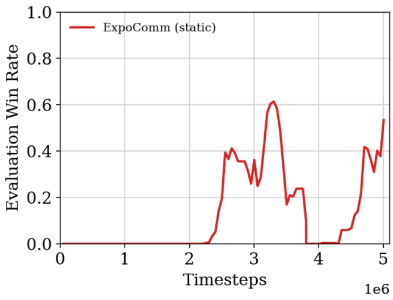
Two sets of Battle runs are reported below. **Run A** is the static-ExpoComm runs in Table 2 and Figure 1. **Run B** is a separate set that trained all five methods we could reproduce (Table 3); its models are also used for the zero-shot transfer experiment (the paper's Figure 5).

Table 2. Evaluation win rate of the learned red team and mean return of static ExpoComm on Battle (seed 0, Run A), over 100 greedy test episodes.

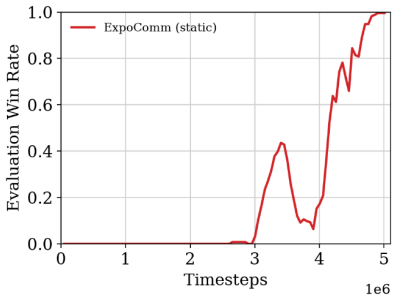
Run	Learned agents	Final win rate	Mean win rate, last 10 evaluations	Final return	Mean return, last 10 evaluations
Battle25	20	0.294	0.222	0.38	0.029
Battle35 (two jobs, joined at 3.8M steps)	42	0.534	0.330	0.41	−0.195
Battle40	64	0.996	0.937	2.12	1.756



(a) Battle w/ 20 agents



(b) Battle w/ 42 agents



(c) Battle w/ 64 agents

Figure 1. Evaluation win rate of static ExpoComm on Battle, seed 0, Run A, 100 test episodes every 50,000 steps: (a) 20, (b) 42 and (c) 64 learned agents. The 42-agent curve joins two jobs at 3.8 million steps.

Win-rate curves. Figure 1 shows the evaluation win rate over training. The 64-agent run rises to close to 1.0 and stays there. The 20-agent run ends at 0.29 and averages 0.22 over its last ten evaluations. Return values in Table 2 are on a much smaller scale; the 20-agent return climbs from about −21 at the first evaluation to about −1 within roughly 300,000 steps and then moves between −0.5 and 0.8 with no clear trend.

The 42-agent run. Reading Figure 1 approximately, the win rate reached about 0.6 near 3.3 million steps, then fell to about 0.2 by 3.5 million and to about 0 around the resume at 3.8 million, stayed near 0 until roughly 4.3 million steps, and then rose to 0.53 by the end. Because the fall began before the resume, it cannot be attributed to the resume alone. The last ten evaluations before the time limit averaged a return of −1.15; after the resume the run ended at a return of 0.41. We cannot say whether the recovery reflects continued learning or an effect of resuming with an empty replay buffer. For comparison, the paper's static ExpoComm curve at 42 agents only starts to rise after about 3 million steps and ends near 0.6, so this run's final value is close to the paper, while its last-ten mean (0.33) is pulled down by the collapse.

Repeated runs. Battle40 was run twice with the same configuration and seed, and the two runs differed noticeably. Table 2 and Figure 1 show the run with the higher win rate. The two Battle25 runs gave identical results.

Training (Run B)

Run B trained five methods: IDQN (no communication), ER with K = 1 and K = $\lceil \log_2 N \rceil$, and ExpoComm with K = 1 and K = $\lceil \log_2 N \rceil$. All five used the same three team sizes, seed (0), training length and test schedule as Run A. As in the paper (§4.1), ExpoComm uses the one-peer exponential graph for K = 1 and the static exponential graph for K = $\lceil \log_2 N \rceil$. ER is not in the released code. So, we implemented it as the paper describes. Each agent receives messages from itself plus K other agents sampled uniformly at random, redrawn every timestep. Each run used one GPU that was checked to be visible to PyTorch before training started. DGN+TarMAC and CommFormer are not in the released code and were not reproduced.

Table 3 gives each method's win rate against the pretrained opponent at the end of training (mean of the last five evaluations). In Figure 4, solid lines are K = 1 and dashed lines are K = $\lceil \log_2 N \rceil$, so static ExpoComm is the dashed red curve.

Table 3. Training win rate against the pretrained opponent after 5 million steps (Run B, seed 0, mean of the last five evaluations; bold = highest). Paper values are the end points of its Figure 4 curves, read by eye; its shaded bands are wide.

Team size	IDQN	ER (K = 1)	ER (K = $\lceil \log_2 N \rceil$)	ExpoComm (K = 1)	ExpoComm (K = $\lceil \log_2 N \rceil$)	Paper, Fig. 4 (approx.)

20 agents	0.40	0.22	0.55	**0.97**	0.77	ExpoComm ≈ 0.6–0.8; ER ≈ 0.55–0.7; IDQN ≈ 0.5
42 agents	0.14	0.48	0.57	**1.00**	0.88	ExpoComm K = 1 ≈ 0.8, static ≈ 0.6; ER ≈ 0.6–0.8; IDQN ≈ 0.45
64 agents	0.13	0.54	0.84	0.98	**1.00**	ExpoComm ≈ 0.85; ER K = 1 ≈ 0.95, static ≈ 0.85; IDQN ≈ 0.45

A communication method is best at every team size, and ExpoComm is best at all three, as in the paper. ExpoComm with K = 1 does as well as or better than the paper's curve. Our IDQN is weaker than the paper's at 42 and 64 agents (0.13–0.14 against about 0.45), which makes the gap to the communication methods larger.

Difference between Run A and Run B

For static ExpoComm, Run B reached 0.77, 0.88 and 1.00 at 20, 42 and 64 agents, at or above the paper's dashed curve. Run A (Table 2) averaged 0.22, 0.33 and 0.94 over its last ten evaluations. The two runs agree at 64 agents but differ widely at 20 and 42 agents, even though they share the configuration and the seed. One possible cause is the hardware the runs used. Run A's logged configuration shows use_cuda: False. The code sets this automatically when it cannot see a GPU, and it then continues on the CPU without warning. We met the same problem on some of the cluster's GPU nodes during Run B. In those cases we excluded the affected nodes and reran on a working GPU. Run A may therefore have trained on the CPU and Run B on a GPU, and the two can give different numerical results from the same seed. Seed-level variation could also explain the gap. However, with one run each, we cannot tell the two apart.(!! this part not sure yet)

Zero-shot transfer (paper Figure 5, p. 9)

Each method is trained with X agents per team (X = 20, 42, 64) and then plays every other method with Y ≥ X agents per team (Y = 20, 42, 81, 121). Each pair plays 200 games, the method with more wins takes the pair, and the bar height (Win Count) is the number of pairs a method wins in that setting. The paper concludes that ER and ExpoComm both transfer well and that ExpoComm transfers better under a small communication budget. In the paper's figure, ExpoComm reaches the maximum win count in every panel.

- With five methods, the maximum win count is 4. The paper has seven methods, maximum is 6.
- At the larger test size, ExpoComm and ER use the test size's budget, K = $\lceil \log_2(N - 1) \rceil + 1$ including the agent itself. This configuration follows the author's release code.
- The authors released no code for head-to-head play, and the paper does not say which side each method plays, whether play is greedy, or which checkpoint is used. We used side balance (100 games per side), 5% random actions ($\varepsilon = 0.05$) and each run's final checkpoint. The random actions are needed because Battle starts from fixed formations, so fully greedy games are exact replays.

- These choices change the result: with the same models at 20 → 20, greedy play gives the most pair wins to ER (K = $\lceil \log_2 N \rceil$) (4 wins), while ε = 0.05 gives them to ExpoComm (K = 1) (3 wins).

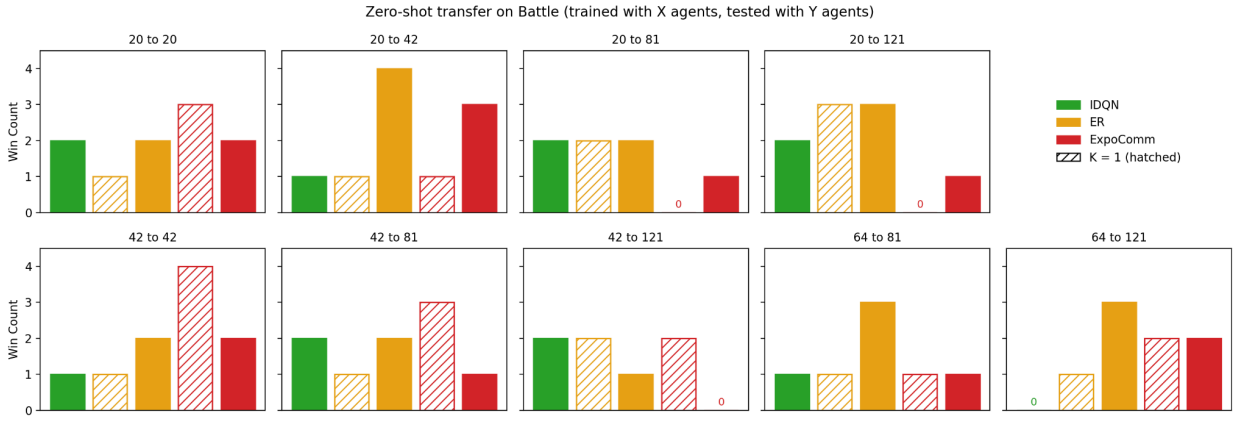


Figure 2. Our reproduction of the paper's Figure 5 (Run B models, seed 0, ε = 0.05, side-balanced, 200 games per pair). Filled bars: K = $\lceil \log_2 N \rceil$; hatched bars: K = 1. Maximum win count is 4.

Table 4. Win counts per setting (pairs won out of 4; bold = highest).

Setting	IDQN	ER (K = 1)	ER (K = $\lceil \log_2 N \rceil$)	ExpoComm (K = 1)	ExpoComm (K = $\lceil \log_2 N \rceil$)	Red side won (of 200)
20 → 20	2	1	2	**3**	2	110–175
20 → 42	1	1	**4**	1	3	137–182
20 → 81	**2**	**2**	**2**	0	1	166–200
20 → 121	2	**3**	**3**	0	1	176–200
42 → 42	1	1	2	**4**	2	107–190
42 → 81	2	1	2	**3**	1	130–200
42 → 121	**2**	**2**	1	**2**	0	168–200
64 → 81	1	1	**3**	1	1	37–200
64 → 121	0	1	**3**	2	2	101–200

- **Trained on 20 agents:** ExpoComm (K = 1) is narrowly best at 20 → 20, but ER (K = $\lceil \log_2 N \rceil$) is best at 20 → 42 and ER wins most pairs at 81 and 121 agents. The paper's result that ExpoComm (K = $\lceil \log_2 N \rceil$) wins every pair in this row is **not reproduced**.
- **Trained on 42 agents:** ExpoComm (K = 1) is the strongest method at 42 → 42 and at 42 → 81. This agrees with the paper that ExpoComm transfers best under small budgets.
- **Trained on 64 agents:** ER (K = $\lceil \log_2 N \rceil$) wins the most pairs in both panels. All communication methods beat IDQN decisively, consistent with the paper, where IDQN wins at most one pair.

Starting-side effect. Every method was trained only as the red team. In head-to-head play between communication methods the red side wins far more often, and the effect grows with team size. At 81 and 121 agents, most pairs finish close to 100:100 because the red side wins almost every game. A pair is often won by only a few games, which is within chance (about ±15 games out of 200), so these panels say little about which method is better.

Transfer against the fixed opponent. To test transfer without any pairing choices, each trained model played the same pretrained opponent it was trained against, but with more agents per team. This removes both the side effect and our head-to-head protocol (Table 5).

Table 5. Win rate against the pretrained opponent when tested on larger teams (Run B models, 200 games each; 95% margin ≈ ±0.07). Bold = best, shown only where the test separates the methods.

Trained → tested	IDQN	ER (K = 1)	ER (K = ⌈$\log_2 N$⌉)	ExpoComm (K = 1)	ExpoComm (K = ⌈$\log_2 N$⌉)
20 → 20	0.51	0.35	0.56	**0.89**	0.67
20 → 42	0.43	0.57	0.68	**0.72**	0.65
20 → 81	0.92	0.75	0.99	0.96	0.96
20 → 121	0.98	0.85	1.00	0.98	1.00
42 → 42	0.41	0.41	0.71	**0.97**	0.56
42 → 81	0.77	0.86	0.94	**1.00**	0.97
42 → 121	0.88	0.92	0.98	1.00	1.00
64 → 64	0.15	0.74	**0.92**	**0.92**	0.91
64 → 81	0.10	0.85	0.98	1.00	0.98
64 → 121	0.14	0.94	0.99	0.99	1.00

Wherever the test separates the methods, ExpoComm (K = 1) is best or tied for best, which supports the paper's claim about transfer under small.

Discussion

With win rate as the metric, the learned team ended well above zero in all three settings: 0.29 with 20 agents, 0.53 with 42 and 0.996 with 64. These values sit broadly within the range of the curves in the paper's Figure 4 (read approximately). The paper uses the static graph for its K = ⌈log2N⌉ budget (§4.1), so static ExpoComm is the dashed red curve, which ends at roughly 0.6–0.8, 0.6 and 0.85 for 20, 42 and 64 agents. Run A is below it at 20 agents, close to it at 42 agents on its final value (but not on its last-ten mean), and at or above it at 64 agents; Run B is at or above it at all three sizes. ~~but we cannot tell which of those curves is static ExpoComm, so we cannot say whether our runs match it.~~ The points below limit what we can conclude.

Return and win rate tell different stories. Return stayed near zero for 20 and 42 agents while the win rate was 0.29 and 0.53, so return is a poor proxy for win rate in this task. The paper

plots win rate and gives no numeric Battle values in the text, so the win-rate columns of Table 2 and 3 are the ones to compare with its figure.

The 20-agent run is the weakest. Its win rate ended at 0.29 and averaged 0.22 over the last ten evaluations, near the low end of the paper's range. We have not yet watched evaluation episodes, so we do not know what the agents do. Recording a greedy episode as a GIF would show it.

Single runs vary. Two runs of Battle40 with the same configuration and seed differed noticeably, and neither was better on both win rate and return, while two runs of Battle25 matched. We do not know why. Run B adds a further comparison: same configuration and seed as Run A, but very different win rates at 20 and 42 agents. The paper averages five seeds, so our curves are best read as individual samples, and the Battle40 row and curve shown are for the run with the higher win rate.

Remaining differences from the paper. The differences we know of are the replay buffer size and the single seed (Table 1). Neither has been tested yet: running seeds 1 and 2 of the smallest setting, and comparing replay buffers of 2000 and 5000 with matched seeds, are the next experiments.

Zero-shot transfer is only partly reproduced. The paper's result that, ExpoComm ($K = \lceil \log_2 N \rceil$) winning every pair when trained on 20 agents, is not reproduced. ER ($K = \lceil \log_2 N \rceil$) wins the most pairs at 20 → 42 and when trained on 64 agents. The head-to-head protocol is our own, and it changes the winners, so part of the gap may come from evaluation rather than from the methods.

Limitations

- **One seed per setting.** The paper uses five seeds with confidence intervals. Our curves have none, so we cannot attach uncertainty to any single number.
- **Spliced and repeated runs.** The 42-agent curve joins two jobs, and the empty replay buffer after the resume may affect the curve just after 3.8 million steps. The two Battle40 runs differ, and the run with the higher win rate is reported.
- **Partial experiment tracking.** The original runs were submitted with Weights & Biases disabled, so their curves were parsed from the printed "Recent Stats" blocks in the training logs. Only the second runs and the resumed 42-agent segment are on W&B, and the logs may omit metrics that the code sends only to W&B.
- **Unverified configuration details.** We have not re-checked the replay buffer size in the released configuration. The logged 20-agent configuration shows `topk_neighbors: 6`, whereas $\lceil \log_2 20 \rceil = 5$, and we have not traced how the configuration name maps to the neighbour set. It also shows `use_cuda: False`, so we do not know whether training used the GPU.

- **Opponent policy.** The blue team follows a pretrained policy shipped with the repository. We did not inspect or retrain it, so we cannot say how strong the opponent is.
- **Comparison with the paper.** We compare the paper's curves by eye from its figure and do not know which curve is static ExpoComm, so the comparison stays qualitative.

Wandb links:

- **Battle 25:** https://forge.coreweave.com/wandb/erikvormala-nus/ExpoComm_Battle25?nw=nwusererikvormala
- **Battle 35:** https://forge.coreweave.com/wandb/erikvormala-nus/ExpoComm_Battle35/runs/l72efw9h?nw=nwusererikvormala
- **Battle 40:** https://forge.coreweave.com/wandb/erikvormala-nus/ExpoComm_Battle40/runs/zwbg5boe?nw=nwusererikvormala
- **Battle, all five methods** (Run B; IDQN, ER and ExpoComm at 20, 42 and 64 agents; used for Figure 2): https://forge.coreweave.com/wandb/wulala379-nus/expocomm-battle
- **IMP, all five methods:** https://forge.coreweave.com/wandb/wulala379-nus/expocomm-imp

Results

Scenario	N	QMIX (K = 0)	ER (K = 1)	ER (K = ⌈log₂N⌉)	ExpoComm (K = 1)	ExpoComm (K = ⌈log₂N⌉)
Uncorrelated	50	24.89 (26.42 ± 3.43)	28.15 (24.91 ± 3.77)	**31.46** (26.62 ± 2.03)	26.94 (27.31 ± 2.26)	28.62 **(28.26 ± 2.51)**
Correlated	50	−27.95 (24.81 ± 4.16)	44.87 (34.63 ± 9.72)	44.01 (34.76 ± 5.07)	43.43 **(43.82 ± 6.33)**	**47.88** (40.01 ± 3.19)
OWF	50	63.12 (62.45 ± 3.46)	60.90 (62.99 ± 3.02)	64.91 (61.70 ± 4.62)	63.78 (64.66 ± 0.26)	**65.03 (65.19 ± 0.51)**
Uncorrelated	100	22.92 (12.86 ± 6.88)	22.31 (21.94 ± 5.97)	−699.06 [1] (18.36 ± 12.92)	18.99 (27.34 ± 13.32)	**24.77 (27.81 ± 5.71)**
Correlated	100	34.20 (−40.20 ± 96.35)	33.62 (−65.14 ± 65.08)	23.07 (9.84 ± 32.27)	29.35 **(19.17 ± 23.94)**	**34.76** (17.25 ± 22.70)
OWF	100	64.75 (65.55 ± 0.53)	64.01 **(66.70 ± 0.50)**	**66.66** (65.92 ± 0.87)	64.00 (65.26 ± 1.34)	66.35 (66.23 ± 0.38)

Table A. Normalised score (%), seed 0, final evaluation. Our value first, the paper's mean ± std over five seeds in brackets. **Bold** in our value = best method in that row in our run; bold blue bracket = best in the paper. [1] Diverged run, see Section Misalignment and Outliers.

Result Analysis

- ExpoComm is the best method in 4 of 6 scenarios in our run (Correlated N = 50, OWF N = 50, Uncorrelated N = 100, Correlated N = 100), against 5 of 6 in the paper. In OWF N = 100 it is a close second (66.35 vs 66.66).
- ExpoComm's own scores match the paper closely, 9 of its 12 cells lie within one std.

Misalignment and Outliers

Because the paper reports a five-seed mean with its standard deviation, a single seed is expected to fall within one standard deviation about two-thirds of the time and within two standard deviations about 95% of the time. In our run, 19 of 30 cells (63%) are within 1 std and 24 of 30 (80%) within 2 std.

Large-looking gaps are mostly in very noisy cells. In Correlated N = 100 our values (23–35) look far from the paper's (−65 to 19), but the paper's own standard deviations there are ±23 to ±96; every one of our values is within 1.5 std.

Our baselines tend to score higher than the paper's (ER at N = 50; QMIX at N = 100). Possible reasons are the reconstructed baseline configurations and our ER implementation details, which the authors did not release. This narrows ExpoComm's margin, which is why it leads in 4 rather than 5 rows.

Two cells in OWF look extreme in standard-deviation terms (ExpoComm K = 1 at N = 50: 3.4 std; ER K = 1 at N = 100: 5.4 std) only because the paper's reported spread there is tiny (0.26 and 0.50); the absolute gaps are 0.9 and 2.7 points.

There are a few outliers in the results that were not expected.

Cell	Ours	Paper	Explanation
Correlated N = 50, QMIX	−27.95	24.81 ± 4.16	12.7 std below. Might be due to an unlucky seed or our reconstructed QMIX configuration (not released by the authors).
Uncorrelated N = 100, ER (K = ⌈log2N⌉)	−699.06	18.36 ± 12.92	Training collapse(next table).

Steps	Q-value (mean)	TD error	Gradient norm	Test return
200k–600k	~0.05	0.05–0.11	0.5–1.6	improving, best ≈ −170 at 450k
800k–1M	~0	1.4 (≈16× up)	110–130 (≈80× up)	collapses (−635 → −3023)
1.2M–1.4M	~0	0.5–0.65	9–15	stuck ≈ −1800
1.6M → end	~10^7	~10^9	inf (overflow)	stuck ≈ −1800

Divergence of this kind is a known failure mode of deep Q-learning (van Hasselt et al., 2018). Large run-to-run variance on IMP has also been reported by the benchmark's authors (Leroy et al., 2023) and is visible in the paper's own Table 1 (standard deviations up to ±96). With a single seed, such a run is not averaged out. We keep the final evaluation value, as for every other cell, rather than selecting a better checkpoint for this run only.